

GitHub 操作マスターへの道【Mac ユーザー専用版】 ～Terminal × VS Code で学ぶ 個人からチーム開発まで～

第1章 : Git と GitHub の基礎

1.1 Git とは？（現場で使われる背景から学ぶ）

Git は、ソースコードやドキュメントの変更履歴を記録・管理できるツールです。

例えば、レポートやプログラムを書いているときに「前のバージョンに戻したい」「誰がどこを修正したのかを確認したい」ということはありませんか？

Git はまさにその「タイムマシン」のような存在で、どのタイミングで何が変更されたのかを記録し、いつでも戻すことができます。

例え : Git は「Google ドライブのバージョン履歴」機能に似ています。ファイルの編集履歴を自動的に記録して、いつでも戻せる点が共通しています。

1.2 GitHub とは？（SNS 的な側面も含む）

GitHub は、その Git の履歴をクラウド上に保存しておけるサービスです。個人開発だけでなく、チーム開発では複数人が同じコードを同時に編集する必要があります。

そのとき GitHub があれば、遠く離れた人とも同じプロジェクトを安全に共同編集できます。さらに、Pull Request 機能を使えば、「変更提案」→「レビュー」→「承認」といった企業で使われる開発フローを体験できます。

補足 : GitHub は SNS のようなフォロー機能や、スター機能などもあり、世界中のエンジニアが自分のコードを公開・共有しています。

1.3 よく使う用語

用語	意味	例え
リポジトリ (Repository)	ソースコードやファイルの保管庫	家の「本棚」や「倉庫」
コミット (Commit)	ファイルの変更を履歴として記録	「上書き保存」 + 「履歴帳」
プッシュ (Push)	ローカルの変更を GitHub にアップロード	クラウドに「提出」
プル (Pull)	GitHub からローカルにダウンロード	「最新版の資料を取り寄せ」
クローン (Clone)	リポジトリを丸ごとコピー	「レシピ本をまるっと自分の家に持ってくる」
ブランチ (Branch)	並行して開発する分岐線	小説の「別エンディング」や「if ルート」
マージ (Merge)	ブランチを統合すること	「分かれて書いた物語を一つにまとめる」
コンフリクト (Conflict)	同じ場所を同時に編集して衝突した状態	同じ原稿に「赤ペン先生が2人」
プルリクエスト (Pull Request)	変更の提案とレビュー依頼	「先生、これ直しました！見てください！」
イシュー (Issue)	問題点ややることのメモ	「やることリスト」や「バグ報告カード」
フォーク (Fork)	他人のリポジトリを自分のコピーとして持つ	「他人の料理を真似して自分流にアレンジ」
README.md	プロジェクトの説明書	「このアプリの取扱説明書」
.gitignore	Git に登録しないファイルを指定	「これは持っていかなくていいリスト」
コミットメッセージ	変更内容の説明文	「この変更、こういう理由だよ！」
main/master ブランチ	中心のブランチ	「作品の本筋ストーリー」
origin	元のリモートリポジトリ名	「GitHub 上の公式倉庫」
HEAD	今見ているコミットの位置	「現在地のポストイット」
stash	一時的に変更を避難させる機能	「作業途中をロッカーにしまう」
revert	過去のコミットを取り消す操作	「前の状態に時間を巻き戻す」
tag	特定のバージョンに名前をつける	「完成版のしおり」
GitHub Actions	自動テストやビルドを行う機能	「家事代行ロボット」

ハンズオン：「Git と GitHub の世界を体験しよう」

内容：

- 自分の Mac の中で「バージョン管理って何？」を体感

操作（ターミナル）：

```
mkdir git-practice
cd git-practice
echo "Gitはすごい!" > git.txt
```

```
[(base) ayumuyoshinaga@Mac-mini ~ % mkdir git-practice
[(base) ayumuyoshinaga@Mac-mini ~ % cd git-practice
[(base) ayumuyoshinaga@Mac-mini git-practice % echo "Gitはすごい">git.txt
```

※エラーが出ないことを確認してから

```
git init
git add git.txt
git commit -m "初めてのコミット"
```

```
[(base) ayumuyoshinaga@Mac-mini git-practice % git init
Initialized empty Git repository in /Users/ayumuyoshinaga/git-practice/.git/
[(base) ayumuyoshinaga@Mac-mini git-practice % git add git.txt
[(base) ayumuyoshinaga@Mac-mini git-practice % git commit -m "初めてのコミット"
[main (root-commit) 592632d] 初めてのコミット
Committer: Ayumu Yoshinaga <ayumuyoshinaga@Mac-mini.local>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly:
```

```
git config --global user.name "Your Name"
git config --global user.email you@example.com
```

After doing this, you may fix the identity used for this commit with:

```
git commit --amend --reset-author
```

```
1 file changed, 1 insertion(+)
create mode 100644 git.txt
```

確認:

※履歴確認

```
git log
```

※状態確認

```
git status
```

```
[(base) ayumuyoshinaga@Mac-mini git-practice % git log
commit 592632d8b3c9d2fe457229b6c09a59b3b577dc9c (HEAD -> main)
Author: Ayumu Yoshinaga <ayumuyoshinaga@Mac-mini.local>
Date:   Wed Aug 6 16:59:29 2025 +0900
```

初めてのコミット

```
[(base) ayumuyoshinaga@Mac-mini git-practice % git status
On branch main
nothing to commit, working tree clean
```

Git のユーザー情報を明示的に設定しよう！

```
git config --global user.name "taro tanaka"
git config --global user.email "t.tanaka@example.com"
```

t.tanaka@example.com は GitHub に登録してるメールアドレスにしとくと、あとで GitHub と綺麗に紐づくぞ！

設定できたか確認

```
git config --global --list
```

```
[(base) ayumuyoshinaga@Mac-mini git-practice % git config --global --list
http.postbuffer=524288000
user.name=yoshiayu
user.email=yoshiayu1977116@icloud.com
```

すでにコミット済みだけど修正したいとき

- 今の1回目のコミットの記録を正しい名前&メールに書き換えたいなら：
- `git commit --amend --reset-author`
- ※ 上書きになるから、今のうちにやるとキレイに履歴が残る！

第2章：Mac での Git・GitHub 環境構築

2.1 Git のインストール確認

ターミナルを開いて以下を実行：

```
git --version
```

```
ayumuyoshinaga@Mac-mini git-practice % git --version  
git version 2.39.5 (Apple Git-154)
```

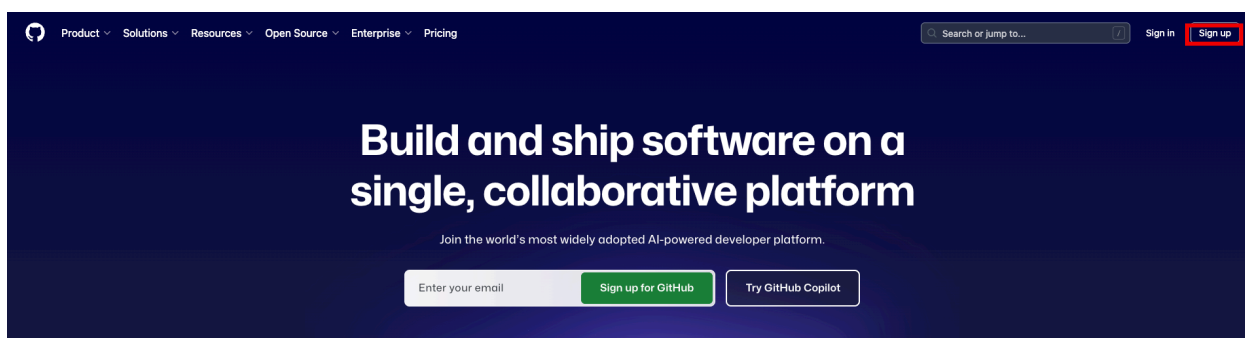
インストールされていない場合：

```
xcode-select --install
```

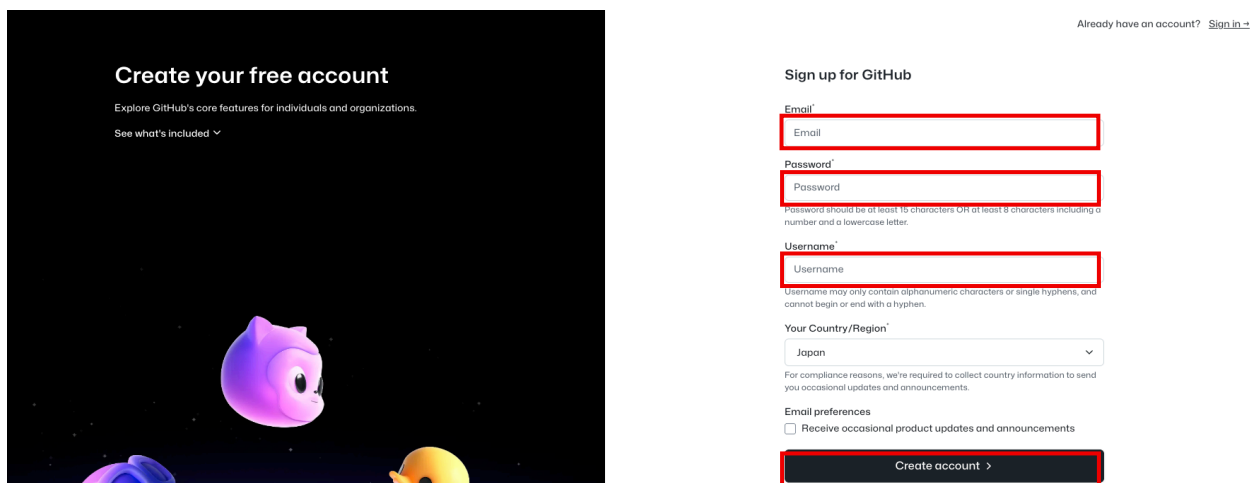
解説：「xcode-select」は Mac の開発者ツールです。Git や C 言語のコンパイラなどが含まれます。

2.2 GitHub アカウントの作成（実習）

1. [GitHub 公式](#)にアクセス



2. 「Sign up」からアカウント登録



3. ユーザー名、メールアドレス、パスワードを入力

4. メール認証を完了する

2.3 SSH キーの生成と設定

SSH って何？

SSH はセキュリティを高めるための「鍵」の仕組みです。GitHub に毎回パスワードを打たずに済むようになります。

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

```
[ayumuyoshinaga@Mac-mini git-practice % ssh-keygen -t ed25519 -C "yoshiayu1977116@icloud.com"]
Generating public/private ed25519 key pair.
Enter file in which to save the key (/Users/ayumuyoshinaga/.ssh/id_ed25519):
/Users/ayumuyoshinaga/.ssh/id_ed25519 already exists.
Overwrite (y/n)? y
[Enter passphrase for "/Users/ayumuyoshinaga/.ssh/id_ed25519" (empty for no passphrase):
[Enter same passphrase again: ]
Your identification has been saved in /Users/ayumuyoshinaga/.ssh/id_ed25519
Your public key has been saved in /Users/ayumuyoshinaga/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256: [REDACTED]@icloud.com
The key's randomart image is:
+--[ED25519 256]--+
|      +==+.+      |
|    ..+ = .       |
|   oo= o          |
|  +o. .           |
| S.. . .o         |
| .. . o.=+        |
| o .o + *.&       |
| . ..o..E X*      |
|   . oBo=         |
+-----[SHA256]-----+
```

```
pbcopy < ~/.ssh/id_ed25519.pub
```

※このコマンドでコピーができていますよ！

1. GitHub → Settings → “SSH and GPG Keys”
2. “New SSH Key” から貼り付ける

Add new SSH Key

Title

※分かりやすい key 名にする

Key type

Authentication Key

Key

Begins with 'ssh-rsa', 'ecdsa-sha2-nistp256', 'ecdsa-sha2-nistp384', 'ecdsa-sha2-nistp521', 'ssh-ed25519', 'sk-ecdsa-sha2-nistp256@openssh.com', or 'sk-ssh-ed25519@openssh.com'

※このまま command + v でコピー

Add SSH key

SSH keys

New SSH key

This is a list of SSH keys associated with your account. Remove any keys that you do not recognize.

Authentication keys



attendance-system

SHA256:ELzJLrLGZRr4jQjiZGSnz09LG8R1B1M8UGwcNLEnc

Added on Jul 17, 2025

Last used within the last 3 weeks — Read/write

SSH

Delete



practice

SHA256:hVFHowT5rNpt95z8U96FPcu5E06KWEgF+LXtzndHbg

Added on Aug 7, 2025

Never used — Read/write

SSH

Delete

Check out our guide to [connecting to GitHub using SSH keys](#) or troubleshoot [common SSH problems](#).

認証テスト:

```
ssh -T git@github.com
```

※初回は yes で入力

第3章：個人リポジトリでの基本操作

3.1 ローカルプロジェクトの作成

ターミナル操作

```
cd ~/Desktop
mkdir my-first-repo
cd my-first-repo
git init
touch README.md
echo "# My First Repo" > README.md
git add .
git commit -m "初回コミット"
```

※git add .は全ファイルを追加、commitで履歴に残します。

```
[ayumuyoshinaga@Mac-mini ~ % cd ~/Desktop ]
[ayumuyoshinaga@Mac-mini Desktop % mkdir my-first-repo ]
[ayumuyoshinaga@Mac-mini Desktop % cd my-first-repo ]
[ayumuyoshinaga@Mac-mini my-first-repo % git init ]
Initialized empty Git repository in /Users/ayumuyoshinaga/Desktop/my-first-repo/.git/
[ayumuyoshinaga@Mac-mini my-first-repo % touch README.md ]
[ayumuyoshinaga@Mac-mini my-first-repo % echo "# My First Repo" > README.md ]
[ayumuyoshinaga@Mac-mini my-first-repo % git add . ]
[ayumuyoshinaga@Mac-mini my-first-repo % git commit -m "初回コミット" ]
[main (root-commit) b1fdfdf] 初回コミット
1 file changed, 1 insertion(+)
create mode 100644 README.md
```


3.2 GitHub リポジトリの作成と連携

1. GitHub 上で "New Repository"
2. プロジェクト名を入力 ※ 統一「my-first-repo」
3. 「Initialize this repository with README」はチェックしない
4. SSH の URL をコピー

3.3 push で GitHub と連携

```
git remote add origin git@github.com:ユーザー名/my-first-repo.git
```

Quick setup — if you've done this kind of thing before

 Set up in Desktop or **HTTPS** **SSH** 

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# my-first-repo" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin git@github.com:yoshiayu/my-first-repo.git
git push -u origin main
```

...or push an existing repository from the command line

```
git remote add origin git@github.com:yoshiayu/my-first-repo.git
git branch -M main
git push -u origin main
```

```
git push -u origin main
```

```
[ayumuyoshinaga@Mac-mini my-first-repo % git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 246 bytes | 246.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:yoshiayu/git-practice.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

`-u` は今後 `git push` とだけ打てば OK になる便利オプションです

第4章：履歴・差分・巻き戻し

4.1 git log：履歴の確認

```
git log --oneline --graph
```

```
[ayumuyoshinaga@Mac-mini my-first-repo % git log --oneline --graph  
* b1fdfdf (HEAD -> main, origin/main) 初回コミット
```

- 簡潔に履歴を見るオプション付き

4.2 git diff：差分の確認

```
git diff
```

- 編集前と後の違いを確認できます。

4.3 git status：状態の確認

```
git status
```

```
[ayumuyoshinaga@Mac-mini my-first-repo % git status  
On branch main  
Your branch is up to date with 'origin/main'.  
  
nothing to commit, working tree clean
```

- 変更済み／未登録ファイルを確認できます

4.4 git restore：変更の取り消し

git restore ファイル名

- 編集をなかったことにする機能

例：ワードの「取り消し」機能に似ています

ハンズオン：「ログや差分を見てみよう」

内容：

- コードをいじって履歴と差分を体感する

操作：

```
echo "新しい行" >> README.md
git diff
```

```
[ayumuyoshinaga@Mac-mini my-first-repo % echo "新しい行" >> README.md
[ayumuyoshinaga@Mac-mini my-first-repo % git diff
diff --git a/README.md b/README.md
index 151e026..3e0155b 100644
--- a/README.md
+++ b/README.md
@@ -1,2 @@
 # My First Repo
+新しい行
```

```
git add README.md
git commit -m "新しい行を追加"
```

確認ポイント：

```
git log --oneline --graph
```

```
[ayumuyoshinaga@Mac-mini my-first-repo % git log --oneline --graph
* c3fcca6 (HEAD -> main) 新しい行を追加
* b1fdfdf (origin/main) 初回コミット
```

第5章：ブランチで安全に開発

なぜブランチを使うのか？

- main ブランチは“製品版”のようなもの。常に安定して動く状態にしておく必要がある。
- 新しい機能や修正をいきなり main でやると、**バグやミスで全体が壊れる危険**がある。
- そこでブランチを作って、**別の場所で安全に実験・開発**できる仕組みが「ブランチ」だ！

例え：

main は「表に出す Web サイト本番版」、
ブランチは「裏でいじってる開発用コピー」

5.1 ブランチの作成と切り替え

```
git checkout -b feature-hello
```

- Git checkout で移動
- -b で feature-hello を作成
- 同時に行えている

```
[ayumuyoshinaga@Mac-mini my-first-repo % git checkout -b feature-hello  
Switched to a new branch 'feature-hello'
```

```
git branch
```

```
[ayumuyoshinaga@Mac-mini my-first-repo % git branch  
* feature-hello  
main
```

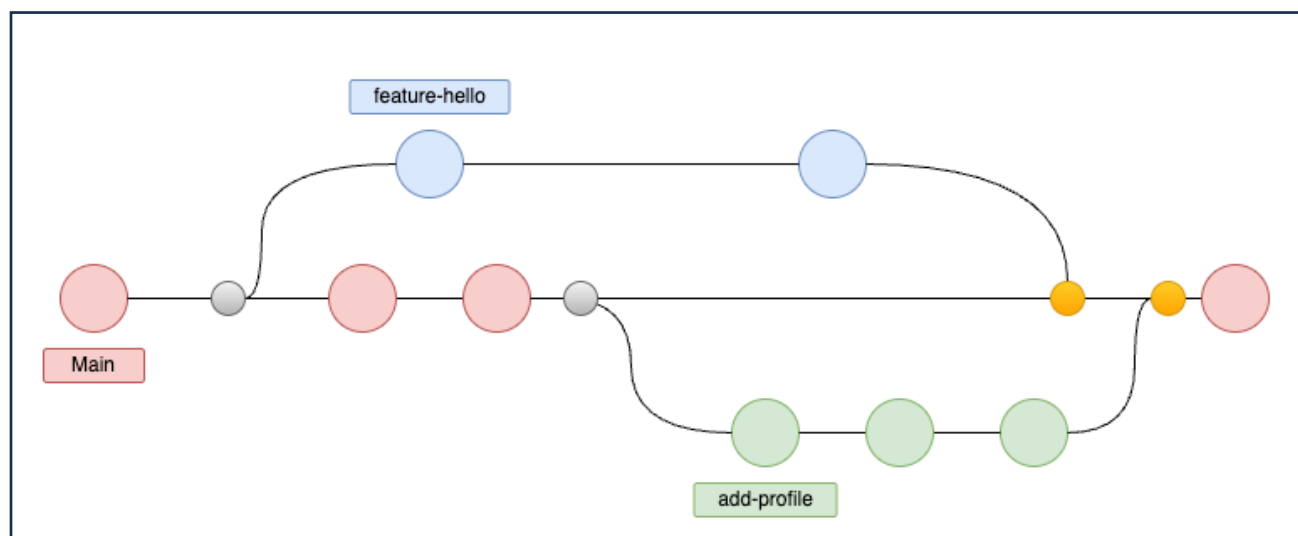
5.2 編集とコミット

```
echo "Hello Git" > hello.txt
git add hello.txt
git commit -m "add hello"
git push -u origin feature-hello
```

```
[ayumuyoshinaga@Mac-mini my-first-repo % echo "Hello Git" > hello.txt
[ayumuyoshinaga@Mac-mini my-first-repo % git add hello.txt
[ayumuyoshinaga@Mac-mini my-first-repo % git commit -m "add hello"
[feature-hello 522afe9] add hello
 1 file changed, 1 insertion(+)
 create mode 100644 hello.txt
[ayumuyoshinaga@Mac-mini my-first-repo % git push -u origin feature-hello
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 10 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (6/6), 554 bytes | 554.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'feature-hello' on GitHub by visiting:
remote:   https://github.com/yoshiayu/git-practice/pull/new/feature-hello
remote:
To github.com:yoshiayu/git-practice.git
 * [new branch]      feature-hello -> feature-hello
branch 'feature-hello' set up to track 'origin/feature-hello'.
```

5.3 GitHub で Pull Request を作成

- GitHub 上で "Compare & Pull Request"
- タイトルと説明を記入
- 「Create Pull Request」ボタン



ハンズオン：「ブランチで安全に変更してみよう」

内容：

- ブランチを作成して並行開発を実感
- add-profile ブランチを作成
- profile.md の中に”## プロフィール”を記述
- profile.md をステージング
- ”プロフィール追加”というコミット名を付加しコミット
- Push する

確認ポイント：

- GitHub で PR 作成 → 内容確認 → Merge まで実施

ローカルで手動マージする場合：

```
git checkout main  
git pull  
git merge feature-hello
```

第6章：チーム開発（2人組）編

6.1 Organization の準備

- GitHub の右上メニュー → “Your organizations” → “New organization”
- KDGT2025-2 で作成
- メール招待で生徒をチームに追加

6.2 チームリポジトリの clone

```
git@github.com:KDGT2025-2/team-repo.git  
cd team-repo
```

6.3 各自がブランチを作って作業

```
git checkout -b feature-member1
```

6.4 レビュー付き Pull Request

- 相手のコードを確認してコメント
 - 指摘があれば修正して再コミット
-

第7章：コンフリクト体験演習

7.1 コンフリクトの発生例

main: 名前 = "田中"

feature-A: 名前 = "山田"

同じ箇所を同時に変更すると Git は「どちらが正しい？」と困惑します

7.2 コンフリクトの表示例

<<<<<< HEAD

名前 = "田中"

=====

名前 = "山田"

>>>>>> feature-A

7.3 解消方法（VS Code を使う）

1. code . でエディタを開く
2. <<<<<<などの記号を削除し、正しい内容を残す
3. 保存し、再コミット

```
git add .  
git commit -m "conflict resolved"  
git push
```

ハンズオン：「コンフリクトを起こして解決せよ」

内容：

- 敢えて衝突させ、VSCode で解消体験

流れ：

1. 複数人が同じファイル（例：team.txt）を編集
2. 片方が main にマージ → もう片方が push でコンフリクト発生
3. VSCode で表示される <<<<<< を編集し、正常化
4. 再コミット・Push

第8章：まとめと応用課題

課題1：ポートフォリオをGit管理

- about.html, contact.html など複数ページを作成
- コミットメッセージに工夫（例：“add contact section”）

課題2：2人組でToDoアプリ開発

- 機能分担：片方はUI、もう片方はロジック
- feature-ui, feature-logic ブランチで開発
- PR をレビューして統合

ハンズオン：「自分だけのポートフォリオ管理」

内容：

- これまで学んだことを使って1人開発を実践

操作：

1. portfolio フォルダを作成し、HTML ファイルを複数作成
2. git add → commit → push
3. GitHub 上で履歴を確認